PES-VCS Submission Report

Student: Saathvik Shenoy (PES1UG24CS394)

Repository: https://github.com/saathviksp/PES1UG24CS394-pes-vcs

This report includes every screenshot requested in the original template README: 1A, 1B, 2A, 2B, 3A, 3B, 4A, 4B, 4C, and the full integration test. The remaining pages attach those screenshots and provide written answers for QS.1, Q5.2, QS.3, Q6.1, and Q6.2.

- 1A ./test objects passing
- 1B object shard layout under .pes/objects
- 2A ./test tree passing
- 2B raw tree object hex dump
- 3A pes init -> pes add -> pes status
- 3B cat .pes/index
- 4A pes log with three commits
- 4B find .pes -type f | sort
- 4C cat .pes/refs/heads/main and cat .pes/HEAD
- Final make test-integration

Required Screenshots

1A. Output of ./test_objects

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ make test_objects
./test_objects
make: 'test_objects' is up to date.
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

I B. find .pes/objects -type f

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

Required Screenshots

2A. Output of ./test_tree

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ make test_tree
./test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function ‘tree_from_index’:
tree.c:105:17: warning: implicit declaration of function ‘object_write’ [-Wimplicit-function-declaration]
  105 |         int r = object_write(OBJ_TREE, data, len, id_out);
      |                 ^~~~~~~~~~~~
In file included from /usr/include/string.h:535,
                 from tree.c:5:
In function ‘strncpy’,
    inlined from ‘tree_from_index’ at tree.c:126:9:
/usr/include/x86_64-linux-gnu/bits/string_fortified.h:95:10: warning: ‘__builtin_strncpy’ output may be truncated copying 255 bytes from a string of length 511 [-Wstringop-truncation]
   95 |   return __builtin___strncpy_chk (__dest, __src, __len,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   96 |                                   __glibc_objsize (__dest));
      |                                   ~~~~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B. xxd of a tree object

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ find .pes/objects -type f | head -1 | xargs xxd | head -20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
```

Required Screenshots

3A. pes init -> pes add -> pes status

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ make clean
make
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
Initialized empty PES repository in .pes/
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  pes.c
  untracked:  index.h
  untracked:  index.c
  untracked:  object.c.save
  untracked:  pes.h
  untracked:  testfile.txt
  untracked:  commit.c
  untracked:  tree.c
  untracked:  commit.h
  untracked:  Makefile
  untracked:  test_objects.c
  untracked:  .gitignore
  untracked:  tree.h
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  README.md
  untracked:  report.pdf
```

3B. cat .pes/index

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776685531 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776685531 6 file2.txt
```

Required Screenshots

4A. pes log with three commits

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
./pes log
Committed: 602d06a7b7ad... Initial commit
Committed: b716509de447... Add world
Committed: 6f5783965966... Add farewell
commit 6f5783965966850d38cf702c7cf4f1d28b46c5d19567d79f66de3f1fd9accfab
Author: PES1UG24CS394 <PES1UG24CS394@pesu.pes.edu>
Date:   1776685588

    Add farewell

commit b716509de447ea018f9fd797d9190985a54cd3e680e9f2bcac387c1565463070
Author: PES1UG24CS394 <PES1UG24CS394@pesu.pes.edu>
Date:   1776685588

    Add world

commit 602d06a7b7ad7da2d2322efc319eb8ec82d9450bf073c7824808b5ea181aee90
Author: PES1UG24CS394 <PES1UG24CS394@pesu.pes.edu>
Date:   1776685588

    Initial commit
```

4B. find .pes -type ‹ I sort

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ ./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
find .pes -type f | sort
Initialized empty PES repository in .pes/
Committed: 113362c98c7a... Initial commit
Committed: 072b79474674... Add world
Committed: 4231fd8f08c9... Add farewell
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/07/2b79474674d35cf5e627816bec92688e34bae12ed2a768e75a8003c7683a08
.pes/objects/11/3362c98c7ad0239bc4e688f9c2e99b9c8e8cf14dc3a7610e4f7745cc4ef218
.pes/objects/42/31fd8f08c9d0eeef79ad81c21bd363abf20e41e5b211baf1d68e120114a2a0
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/a6/882ee4afdc855ea4e39f4e0fc77d90e73a6f72deb383e11934310a10a030f6
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/refs/heads/main
```

Required Screenshots

4C. refs/heads/main and HEAD

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ cat .pes/refs/heads/main
cat .pes/HEAD
4231fd8f08c9d0eeef79ad81c21bd363abf20e41e5b211baf1d68e120114a2a0
ref: refs/heads/main
```

Final. make test-integration

```
saaths@saaths-VirtualBox:~/PES1UG24CS394-pes-vcs$ make clean
make
make test-integration
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: e8185bc8cf87... Initial commit

Log after first commit:
commit e8185bc8cf87f4bf4f9e6680cb676ccd58f866ed5ada19778559c3d1314a2635
Author: PES1UG24CS394 <PES1UG24CS394@pesu.pes.edu>
Date:   1776685600

    Initial commit


--- Second Commit ---
Committed: 95dbf75dfc05... Update file.txt

--- Third Commit ---
Committed: c5d55e9a51d6... Add farewell

--- Full History ---
commit c5d55e9a51d68247bfb8dc688aea11ba304f95f9e9324d3ef1ed1865aa5ee6d4
Author: PES1UG24CS394 <PES1UG24CS394@pesu.pes.edu>
Date:   1776685601

    Add farewell

commit 95dbf75dfc05fa80b310425ff88e5041ef1bff6fc6fc3b9a873d02a5009262bc
Author: PES1UG24CS394 <PES1UG24CS394@pesu.pes.edu>
```

```
    Update file.txt

commit e8185bc8cf87f4bf4f9e6680cb676ccd58f866ed5ada19778559c3d1314a2635
Author: PES1UG24CS394 <PES1UG24CS394@pesu.pes.edu>
Date:   1776685600

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
c5d55e9a51d68247bfb8dc688aea11ba304f95f9e9324d3ef1ed1865aa5ee6d4

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/95/dbf75dfc05fa80b310425ff88e5041ef1bff6fc6fc3b9a873d02a5009262bc
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c5/d55e9a51d68247bfb8dc688aea11ba304f95f9e9324d3ef1ed1865aa5ee6d4
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/e8/185bc8cf87f4bf4f9e6680cb676ccd58f866ed5ada19778559c3d1314a2635
```

Analysis Answers

Q5.1

To implement pes checkout <branch>, I would first read .pes/refs/heads/<branch> to find the target commit hash, then update .pes/HEAD so it points to that branch name. After that, I would load the target commit, read its root tree, recursively materialize every blob into the working directory, create missing directories, and remove tracked files that are present in the current branch but absent from the target tree. The operation is complex because checkout must coordinate HEAD, branch refs, the index, and the working tree together while refusing to overwrite user changes or leave the repository in a partially switched state.

Q5.2

Dirty-working-directory conflicts can be detected by comparing the current working tree against the index before checkout begins. For each tracked index entry, stat the working file and compare its mtime and size to the cached metadata; if they differ, recompute the blob hash and compare it to the staged hash. Then compare the current branch tree and target branch tree for the same path. If the file is modified locally and the target branch would also change or delete that path, checkout must stop with a conflict instead of overwriting the file.

Analysis Answers

Q5.3

In detached HEAD state, HEAD contains a commit hash directly instead of a branch ref, so new commits still get created but no branch name moves forward to reference them. Those commits become easy to lose once the user switches away, because no ref guarantees they remain reachable. Recovery is possible as long as the hash is known, for example from log output or a reJlog-like mechanism; the user can create a new branch or ref that points to the detached commit chain and make it reachable again.

Q6.1

Garbage collection should begin from every reachable root: all files under .pes/refs/heads plus any hash stored directly in HEAD when detached. Starting from those commit hashes, traverse each commit, mark its tree, then recursively mark every subtree and blob referenced by those trees. A hash set is the right structure for reachability because membership checks must be O(1) while walking potentially hundreds of thousands of objects. With 100,000 commits and 50 branches, the walk still visits each reachable object at most once; in the worst case that means roughly every live commit plus every live tree and blob, so the total number of visited objects can easily be several hundred thousand or more depending on repository size.

Analysis Answers

Q6.2

Running GC concurrently with commit creation is dangerous because commit creation writes blobs and trees before the final commit object and branch ref become visible. A concurrent GC might scan refs, fail to see the new commit yet, and conclude that one of those freshly written blobs or trees is unreachable, deleting it just before the new commit records its hash. That would leave the new commit pointing at missing objects. Git avoids this race by coordinating GC with repository locks and by treating recently created objects specially so they are not collected until the new refs and packfiles are safely published.